Roop Shree Construction — Website Implementation Plan

For: Claude CLI build execution Stack: PHP 8.3 + PostgreSQL + HTML/CSS/JS (Bluehost Shared Prime hosting)

Confirmed Hosting Details

- Bluehost Shared Hosting — Prime level
- Domain: roopshreeconstruction.com (live, already pointed, currently running WordPress — to be fully replaced)
- PHP 8.3 (ea-php83) confirmed via MultiPHP Manager
- Database: PostgreSQL (available via phpPgAdmin / PostgreSQL Database Wizard in cPanel) — chosen over MySQL
- File Manager, FTP, Git Version Control all available in cPanel
- No Node.js support on this plan — confirms PHP as backend

Build Strategy: Local First

1. Build and fully test the entire site **locally** (local PHP + PostgreSQL environment) before touching the live Bluehost server.
2. Only once local build is approved and tested end-to-end:
   - Decide WordPress removal approach (likely: build into a fresh subfolder or staging area first, or clear `public_html` directly — decide at deploy time)
   - Create live PostgreSQL DB + credentials via cPanel PostgreSQL Wizard
   - Upload via File Manager/FTP or Git
3. This avoids breaking the live domain during development.

Local Environment Requirements (for Claude CLI)

- PHP 8.3 (match production)
- PostgreSQL (local instance — via Docker, Postgres.app, or native install)
- Local PHP built-in server (`php -S localhost:8000`) is sufficient for dev/testing — no need for Apache/Nginx locally unless replicating `.htaccess` behavior needs testing
- `.env` or a local-only `db.php` config (gitignored) with local DB credentials, separate from a production config template

---

1. Project Overview

Informative real estate website for Roop Shree Construction (Jodhpur) with:

- Static public pages (Home, About, Contact) — not editable by client
- Dynamic Properties system (list + detail pages) — pulled from DB

- `/admin` panel — client (site owner's brother) can add/edit/delete property listings only. No access to site structure, About, Contact, or design.

Design direction: inspired by ashapurna.com layout (hero → featured grid → detail pages) but with distinct styling — underline hover effects on buttons/links, custom color palette and typography (final assets/images provided by client).

---

2. Folder Structure

```
/public_html
  /assets
    /css
      style.css
      admin.css
    /js
      main.js
      admin.js
    /images
      (static site images: logo, hero, icons)
  /uploads
    /projects
      /{project_id}/
        images/
        brochure.pdf
  index.php              (Home)
  properties.php         (All Properties listing, filter by type/status)
  property.php           (Single property detail, ?id=)
  about.php
  contact.php
  contact-handler.php    (PHP mail script for contact form POST)
  /admin
    index.php            (Login page)
    logout.php
    dashboard.php        (List all projects, edit/delete actions)
    add-project.php      (Add form)
    edit-project.php     (Edit form, ?id=)
    delete-project.php   (POST handler, confirm + delete DB row + files)
    /includes
      auth-check.php     (session guard, include at top of every admin page)
      db.php             (DB connection config)
  /includes
    header.php
    footer.php
    db.php               (shared DB connection for public pages)
  /sql
    schema.sql           (DB schema, run once via phpMyAdmin)
```

3. Database Schema (PostgreSQL)

```sql
```

```sql
CREATE TYPE project_type AS ENUM ('Flat','Plot','Villa','Commercial');
CREATE TYPE project_status AS ENUM ('Available','Sold','Coming Soon');

CREATE TABLE admin_users (
  id SERIAL PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE projects (
  id SERIAL PRIMARY KEY,
  title VARCHAR(150) NOT NULL,
  type project_type NOT NULL,
  location VARCHAR(150),
  price VARCHAR(100),           -- store as text: "₹18.31 Lakh onwards" (flexib
  size VARCHAR(100),            -- e.g. "1200 sq. yard"
  status project_status DEFAULT 'Available',
  rera_number VARCHAR(100),
  short_desc VARCHAR(300),      -- card preview text
  full_desc TEXT,               -- detail page content
  featured BOOLEAN DEFAULT FALSE,
  brochure_path VARCHAR(255),   -- path to uploaded PDF
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE project_images (
  id SERIAL PRIMARY KEY,
  project_id INT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  image_path VARCHAR(255) NOT NULL,
  sort_order INT DEFAULT 0
);

-- updated_at auto-update trigger (Postgres has no native ON UPDATE CURRENT_TIM
CREATE OR REPLACE FUNCTION set_updated_at() RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = CURRENT_TIMESTAMP;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_projects_updated_at
```

```
BEFORE UPDATE ON projects
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

Seed `admin_users` manually once with a bcrypt-hashed password (generated via PHP `password_hash()`, not typed in plaintext anywhere).

---

4. Build Phases (sequential, for CLI execution)

Phase 1 — Environment & Skeleton

- Confirm PHP version + MySQL availability (pending Bluehost cPanel check)
- Create folder structure above
- Set up `db.php` with placeholder credentials (to be filled after hosting confirmed)
- Create `schema.sql`, do not auto-run — flag for manual execution via phpMyAdmin

Phase 2 — Static Public Pages

- Build `header.php` / `footer.php` partials (nav, logo, footer links, social icons, contact info — matching Ashapurna's footer pattern but restyled)
- Build `index.php` (Home): hero section, "Featured Projects" (query `featured = 1`), CTA buttons with underline-hover effect
- Build `about.php`, `contact.php` (static content, form posts to `contact-handler.php`)
- Build `contact-handler.php`: PHP `mail()` or SMTP (Bluehost supports basic `mail()`; recommend PHPMailer + SMTP if deliverability matters — flag as decision point)
- Build core `style.css`: color variables, typography, button underline-hover animation, responsive grid for property cards

Phase 3 — Dynamic Properties (Public)

- `properties.php`: query all projects, render as cards (image, title, location, price, type badge, status badge), filter buttons (All/Flat/Plot/Villa/Commercial) via JS (client-side filter, no reload)
- `property.php?id=`: full detail — image gallery (simple JS slider/lightbox), full description, size, RERA number, price, "Download Brochure" button (links to PDF), "Enquire" button (mailto or links to contact page with pre-filled project name via query param)

Phase 4 — Admin Auth

- `admin/index.php`: login form → verifies against `admin_users` table using `password_verify()`, starts PHP session on success
- `admin/includes/auth-check.php`: checked at top of every protected admin page; redirects to login if no valid session
- `admin/logout.php`: destroys session

- Basic brute-force protection: limit login attempts (simple session/IP-based counter is enough at this scale)

Phase 5 — Admin CRUD

- `admin/dashboard.php`: table of all projects (title, type, status, featured toggle, edit/delete buttons)
- `admin/add-project.php`:
  - Form fields per schema (title, type, location, price, size, status, RERA, short_desc, full_desc, featured checkbox)
  - Multi-image upload input + single PDF upload input
  - On submit: insert into `projects`, handle file uploads to `/uploads/projects/{id}/`, insert rows into `project_images`
  - **Image handling**: auto-resize/compress on upload (GD library, resize to max width e.g. 1600px, convert to JPG quality ~80) so client can't upload huge files
  - Validate file types strictly (images: jpg/png/webp only; brochure: PDF only) and file size limits
- `admin/edit-project.php?id=`: same form pre-filled, allows replacing/removing individual images, replacing PDF, updating any field
- `admin/delete-project.php`: confirm dialog (JS), then POST deletes DB row (cascades to `project_images`) + deletes actual files from `/uploads/`

Phase 6 — Polish & Hardening

- Mobile responsiveness pass across all pages
- Form validation (client-side JS + server-side PHP) on both public contact form and admin forms
- Sanitize all DB inputs (prepared statements / PDO throughout — no raw SQL concatenation, anywhere)
- Escape all output (`htmlspecialchars`) to prevent stored XSS via admin-entered text fields
- `.htaccess` in `/admin` and `/uploads` as extra layer (deny direct PHP execution in uploads folder; force HTTPS site-wide)
- 404 page, basic SEO meta tags per page (title/description — client can't edit these, hardcoded by you per page)

Phase 7 — Future (not in this build, flagged for later)

- Booking/site-visit scheduling system (separate table: `bookings`, tied to `projects`, admin-side calendar/list view)
- Possibly WhatsApp click-to-chat button (static `wa.me` link — trivial to add anytime)

---

5. Security Notes (non-negotiable, apply throughout)

- All DB queries via PDO (`pdo_pgsql` driver) with prepared statements — no exceptions
- Passwords: `password_hash()` / `password_verify()` only — never plaintext, never MD5/SHA1
- Session-based admin auth, `session_regenerate_id()` on login
- File upload validation: check MIME type + extension, re-encode images via GD (strips malicious payloads hidden in image files), store PDFs outside web-executable context if possible or block script execution in `/uploads` via `.htaccess`
- All admin form inputs escaped on output to prevent XSS
- HTTPS enforced (Bluehost free SSL — force via `.htaccess` redirect)

---

6. Open Decisions (pending your input)

1. ~~Bluehost plan tier + PHP version~~ — CONFIRMED (Shared Prime, PHP 8.3)
2. ~~Database engine~~ — CONFIRMED (PostgreSQL)
3. Admin credentials (username + password — to be generated, hash stored in DB, never in code as plaintext) — decide at deploy stage
4. Contact form delivery method: basic PHP `mail()` vs PHPMailer+SMTP (SMTP more reliable, slightly more setup)
5. WordPress removal approach on live server — decide at deploy stage (clear `public_html` vs stage in subfolder first)
6. Final content: logo, brand colors/fonts, property data, images, PDFs (client to provide)

---

7. Suggested Build Order for Claude CLI Session

Stage A — Local Build & Testing (current stage)

1. Scaffold folder structure + `schema.sql` (PostgreSQL)
2. Set up local PHP 8.3 + PostgreSQL dev environment
3. Shared `db.php` (local config, gitignored), `header.php`, `footer.php`
4. Static pages (Home, About, Contact) + core CSS (get visual style approved before continuing)
5. Properties list + detail pages (with dummy/seed data first)
6. Admin login + auth guard
7. Admin CRUD (add/edit/delete projects, image + PDF upload with compression)
8. Hook public pages to live DB data (remove dummy data)
9. Security pass + responsive/polish pass
10. Full local end-to-end test (add/edit/delete a project, upload images + PDF, view on public site, contact form)

Stage B — Deployment (only after local build is approved)

1. Decide + execute WordPress removal / staging approach
2. Create live PostgreSQL DB + user via cPanel PostgreSQL Wizard, generate production `db.php` config
3. Generate admin credentials, seed `admin_users` table on live DB
4. Upload site files via File Manager/FTP/Git
5. Configure `.htaccess` (HTTPS redirect, uploads folder protection)
6. End-to-end test on live domain
7. DNS/SSL check, final go-live